

BankV3：DAO 治理 金融系統

Decentralized Finance

報告人：羅暉嫻

01 | 選定主題的動機

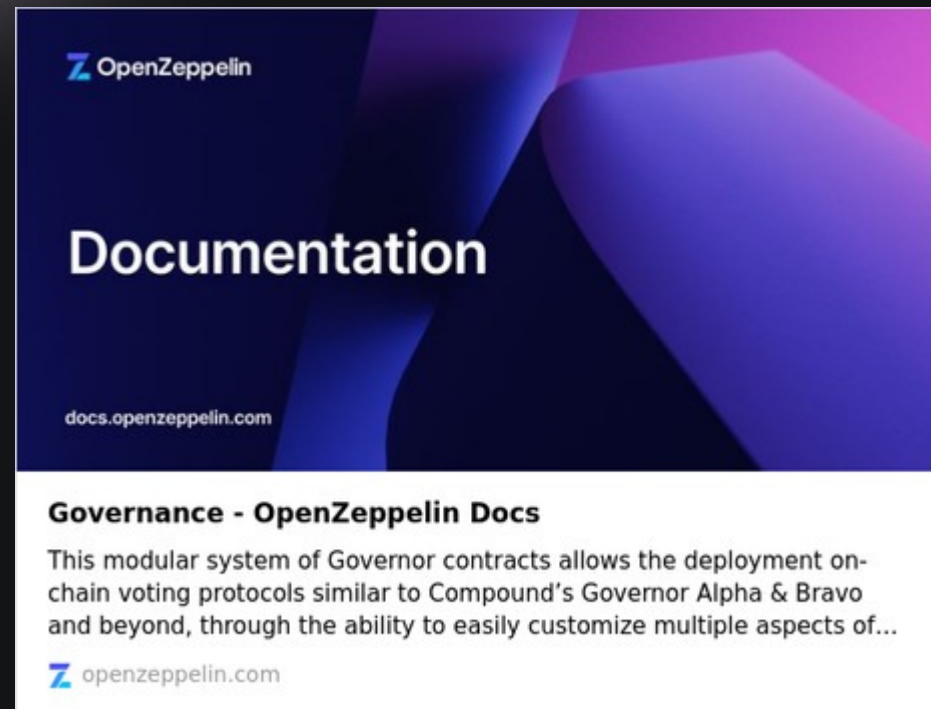
我在之前的課堂報告中介紹過 DeFi 的相關內容，當時對去中心化借貸、清算機制有了一些初步了解。

後來進一步發現這些系統其實可以結合 DAO 治理，讓整個平台的參數調整與決策過程更去中心化。

因此這次的專題希望延續那個主題，實際動手開發一個簡化版的去中心化借貸平台，

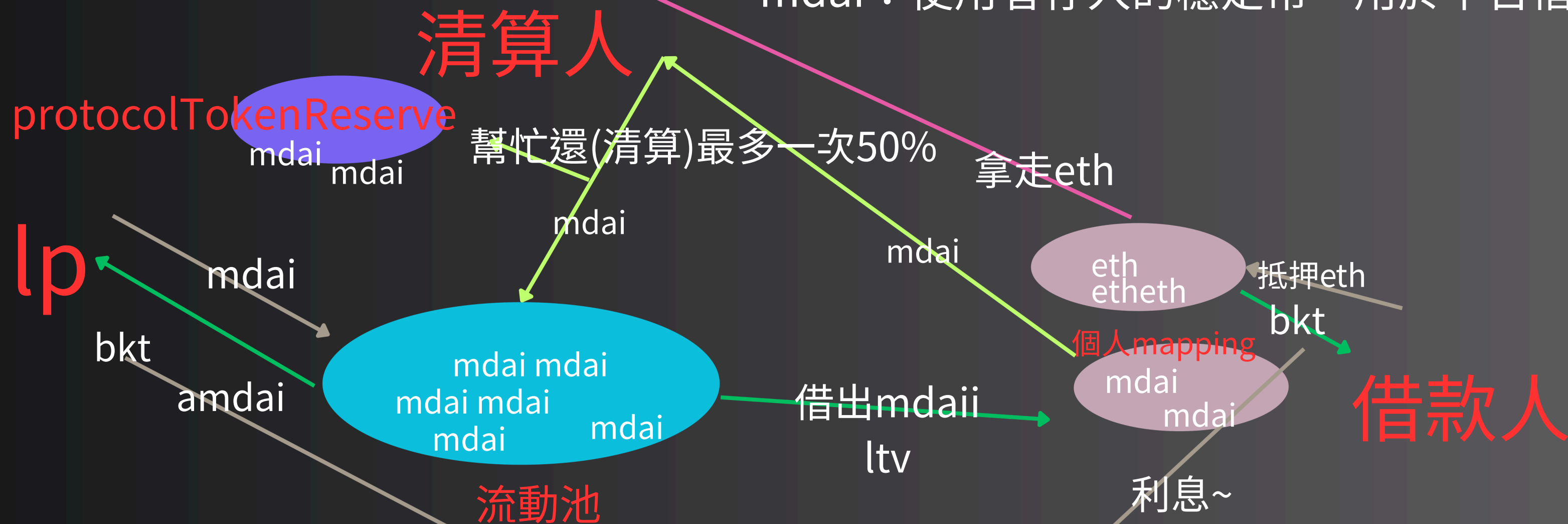
參考的資料

- chatgpt
- Aave、Compound 等主流 DeFi 借貸協議的設計架構
- OpenZeppelin Contracts 4.9.3 的治理模組說明 (Governor + Timelock)
- Hardhat 的用法
- DAO 治理機制與實作流程 (提案、投票、執行)



03 | 使用情境描述

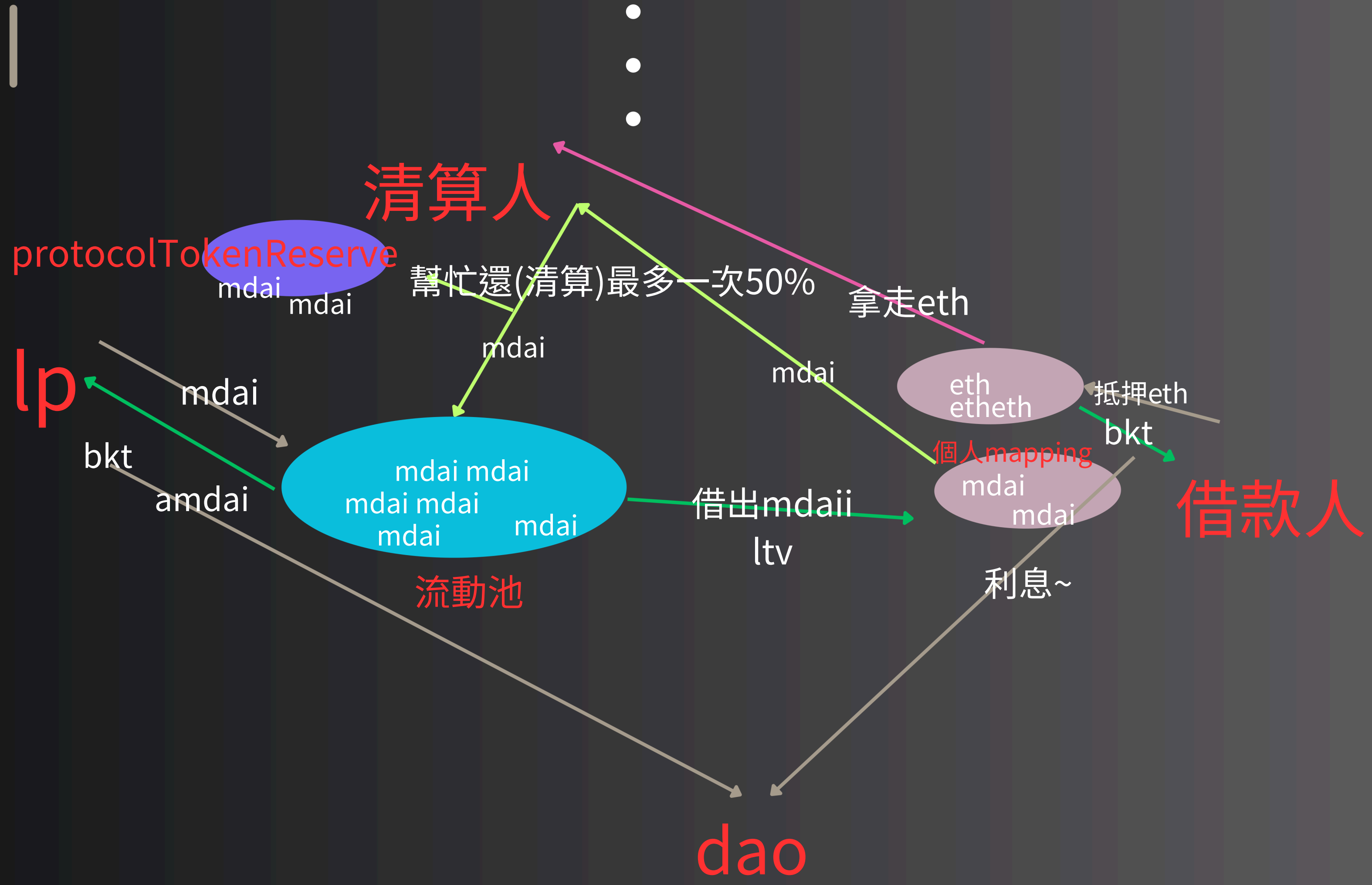
- bkt : 治理代幣獎勵，dao治理權根據
- amdai : 拿回流動性的憑證，兌換憑證
- mdai : 使用者存入的穩定幣，用於平台借貸資金池

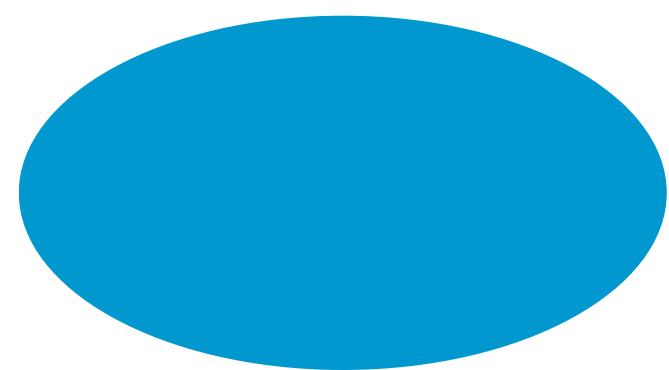


```
mapping(address => RewardInfo) public lpRewards;  
struct RewardInfo {  
    uint256 lastUpdate; // 上次更新時間  
    uint256 unclaimedBKT; // 累積的可領獎勵  
    uint256 supplyAmount; // 存入資產量 (mDAI or ETH)  
}
```

```
mapping(address => uint256) public collateral;  
mapping(address => DebtInfo) public debts;
```

dao





04 | 功能規劃



智能合約部分

- BankV3: 借貸與清算邏輯核心合約
- BankGovernor: 提案治理模組
- BankTimelock: 延遲執行的 Timelock 控制器
- PriceOracle: 模擬 ETH 價格，可調整餵價
- BankToken: dao參與自治的資格票
- MiniDAI: 穩定幣，用於借貸
- aMDAI: LP 憑證代幣，代表供應流動性的權益

import 套件

```
'Governor.sol';
extensions/GovernorCountingSimpl
extensions/GovernorVotes.sol";
extensions/GovernorTimelockContr
/extensions/ERC20Votes.sol";
```

- TimelockController.sol
- ERC20Votes.sol
- ERC20.sol
- ERC20Burnable.sol

項目	價格來源chainlink 實際情況	實驗展示困難
更新頻率	自動從多預言機抓取 ETH/USD 價格	無法人為控制、無法快速下跌
示範可控性	不穩定、難預測、幾小時法配更新	
	無法模擬特定情境（如閃崩）	

```
pragma solidity ^0.8.20;
```

```
/// @title 可調整 ETH 價格的 Oracle，用於測試清算機制
```

```
contract PriceOracle {
```

```
    uint256 private price = 1000 * 1e8;
```

```
    /// @notice 設定新的 ETH 價格 (單位為 8 decimals)
```

```
    /// @param _price 例如輸入 1600 * 1e8 代表 1600 美元
```

```
    function setPrice(uint256 _price) external {
```

```
        price = _price;
```

```
    }
```

```
    /// @notice 查詢當前 ETH 價格 (8 decimals)
```

```
    function getLatestETHPrice() external view returns (uint256) {
```

```
        return price;
```

```
    }
```

```
}
```

```
function getLatestETHPrice() public view returns (uint256) {  
    (, int256 price,,) = priceFeed.latestRoundData();  
    return uint256(price) * 1e10; // 轉成 18 decimals  
}
```

04

功能規劃

權限總覽

【部署者】



```
await bankToken.transfer(timelockAddress, hre.ethers.parseUnits("600000", 18)); // 60%
await bankToken.transfer(bankAddress, hre.ethers.parseUnits("200000", 18)); // 20%
await bankToken.transfer(deployer.address, hre.ethers.parseUnits("200000", 18)); // 20%
console.log("** Initial BKT distributed: 60% to DAO, 20% to Bank, 20% to deployer.");
```

【DAO 治理系統】



- = 主合約邏輯
- = 支援合約 / Token
- = DAO 治理模組

```
const proposalThreshold = hre.ethers.parseUnits("0.1", 18);
const quorumPercent = 2;
```

主要資產：ETH（抵押）、mDAI（借貸流通幣）、aMDAI（LP 憑證） bkt

基礎功能：

- 存款 / 借貸 / 還款 / 清算
- 抵押物管理與借款利率計算
- 借款健康因子（Health Factor）與清算條件判斷
- 流動性池設計（MiniDAI 池) aMDAI LP 代幣
- 協議利息抽成與協議儲備金
- BKT 代幣發行與每日獎勵上限控制

DAO 治理功能：

- BankGovernor 提案與 Timelock 延遲執行控制
- 可調參數（如 interestRate, rewardRatio, dailyCap, reserveFactor 等）
- 使用 ERC20Votes 實作投票權重與委託功能

三大主流程：用戶存 ETH 抵押 → 借出 mDAI → 還款 + 清算邏輯

LP 存入 mDAI → 獲得 aMDAI → 可贖回

DAO 治理調整參數（如利率、獎勵比等）並控制治理權限

功能分類

一般動作 (normal user)

`depositCollateral()` : 存入 ETH 抵押品

`borrow(uint256 amount)` : 用戶根據抵押品價值借出 mDAI

`withdrawCollateral(uint256 amount)`

`repay(uint256 amount)`

`_updateDebt(msg.sender)` 借款前 提領抵押前 還款後 清算前 查詢時

`_updateReward(msg.sender, false)` 抵押時 提領前 存入流動性時 贖回流動性前 領獎勵前

清算人

`liquidate(user, repayAmount)`

Health Factor < 閾值

不能超過 50% 本金或 ETH 可換金額

算 bonus 後換成 ETH 抵押物

本金減少、利息分給 LP + 協議，ETH 給清算人

功能分類

我前端只用了這幾個

請選擇函式

請選擇函式

setInterestRate

setRewardRatio

setDailyCap

setReserveFactor

setProposalThreshold

setQuorumPercent

流動性提供者（LP） LP 存入 mDAI 到池子，換到等量的 aMDAI 憑證

supply(uint256 amount)

redeem(uint256 amount)

dao

```
//-----dao-----
function setInterestRate(uint256 newRate) external onlyGovernance { ...
}
function setRewardRatio(uint256 newRatio) external onlyGovernance { ...
}
function setDailyCap(uint256 newCap) external onlyGovernance { ...
}
function setReserveFactor(uint256 newFactor) external onlyGovernance { ...
}
function setMaxTotalDebt(uint256 newMax) external onlyGovernance { ...
}
//還沒用dap
function setGovernance(address newGov) external onlyGovernance { ...
}
//還沒用dap
function lockGovernance() external onlyGovernance { ...
}
//還沒用dap
function changeStatus(Status _status) external onlyGovernance { ...
}
//治理者（DAO） 提領「協議的收益」
//(more)把 protocolTokenReserve 定期「再注入流動性池」
function injectProtocolEarningsToLP(uint256 amount) external onlyGovernance { ...
}
function withdrawETHReserve(address to, uint256 amount) external onlyGovernance nonReentrant { ...
}
function withdrawTokenReserve(address to, uint256 amount) external onlyGovernance nonReentrant { ...
}
function injectProtocolEarningsToLP(uint256 amount) external onlyGovernance { ...
}
function transferGovernance(address newGov) external onlyGovernance { ...
}
```

05 | 開發過程

Remix 初期實作（概念驗證）

MiniDAI.sol：ERC20 穩定幣合約

Bank.sol：提供抵押 ETH 借出 mDAI 的功能

初始功能包含：depositCollateral()、borrow()、repay()、withdrawCollateral()

前端撰寫

簡單的 DeFi 借貸平台

Hardhat 架構建置

設計 aMDAI 作為 LP 憑證代幣，支援流動性供應與利潤分潤

LP 功能設計加強，清算功能除錯

部署腳本撰寫：使用 scripts/deploy.js 建立自動部署流程

前端撰寫

DAO 治理系統導入

ERC20Votes 代幣：BankToken

TimelockController：延遲執行與權限控管

BankGovernor：提案、投票、執行（使用 OpenZeppelin Governor）

- 被治理函式：如 setInterestRate、setDailyCap、

viewfunction getfunction

問題一：

被清算者hf一定會變高嗎：



Aave 的範例（真實行為）：

借 8000 USDC，抵押 6 ETH（當 ETH = \$2000 時）

ETH 跌到 \$1000， $HF < 1$

清算人還 1000 USDC，獲得價值 1050 的 ETH

被清算人損失抵押品多於債務減少 → HF 下降

在 Aave 的官方文檔中也有提到：

"Partial liquidations will only reduce the user's debt, but their health factor may still stay low or even decline if the **removed collateral exceeds the benefit.**"

可能讓 HF 下降，這是正常的，因此可以一直清算同個人，所以說被清算是很危險的

我遇到的問題 精度!!!!!!

```
- Total Debt: 8000.004629629629629629
- Repay Amount: 300.0
- Max Borrowable: 4800.0
- HF: 0.7499
(ingetHealthFactorfun)uint256 debt = _accruedDebt(user); 8000009259259259259259
- (ingetHealthFactorfun)collateral[user]: 60000000000000000000
- (ingetHealthFactorfun)DECIMALS: 10000
- (ingetHealthFactorfun)Debt: 8000009259259259259259
- (ingetHealthFactorfun)getCollateralValueUSD: 600000000000000000000000
[liquidate] repayAmount: 300000000000000000000000
[liquidate] Bank mDAI: 200000000000000000000000
[liquidate] msg.sender: 0x3c44cdddb6a900fa2b585dd299e03d12fa4293bc
[liquidate] collateral[user]: 600000000000000000000000
if(repayAmount > maxRepay){repayAmount = maxRepay }result 571428571428
[liquidate] maxUSD: 60000000000000
[liquidate] maxRepay: 571428571428
[liquidate] totalUSD: 599999999999
ethPrice: 1000000000000
+++++seized: 59999999999900000000
  bonusETH = seized * BONUS_PERCENT / (BONUS_PERCENT + (DECIMALS - BONUS_PERCENT) 299999999999500000
[liquidate] protocolETH: 5699999999990500000
+++++collateral[user]: 100000000
Final seized: 59999999999900000000
Final bonusETH: 2999999999995000000
Final protocolETH: 5699999999990500000
Remaining collateral: 100000000
New principal: 8000009258687830687831
New collateral[user]: 100000000
(ingetHealthFactorfun)uint256 debt = _accruedDebt(user); 8000009258687830687831
- (ingetHealthFactorfun)collateral[user]: 100000000
New HF: 0
  Liquidation success!
(ingetHealthFactorfun)uint256 debt = _accruedDebt(user); 8000009258687830687831
- (ingetHealthFactorfun)collateral[user]: 100000000
☑ 清算後 HF: 0.0000
  After liquidation:
- Remaining collateral: 0.0000000000001
- Protocol ETH Reserve: 5.69999999999905
- Bonus ETH Received: 0.299818541203188902
- mDAI Spent: 0.000000571428571428
```

單位換算出錯導致清算金額異常

→ 改用 parseUnits、明確使用 1e8 / 1e18 區分

DAO 提案執行卡住 區塊累積不夠：

descriptionHash 不一致 → 寫好統一格式與 encode 流程

各種前端錯誤 前端沒辦法同步更新

Stack too deep 編譯錯誤

用 viaIR: true 或將邏輯函式分拆模組化

感想

這次專案真的讓我收穫很多，尤其是在 DAO 的治理流程和智能合約的整合上。有別於單純寫借貸邏輯，這次還要考慮如何讓參數可以被 DAO 提案調整，像是利率、每日代幣上限、清算比率等等，過程中學會了不少關於 OpenZeppelin 的 Timelock 和 Governor 套件的使用方式。

在開發過程中也遇到不少 bug，像是單位換算搞錯、提案執行卡住、利息拆分不準等等，這些都讓我更深入去理解 Solidity 中數字的精度、以太坊的資料儲存與安全考量。

我也寫了完整的前端頁面，串接 MetaMask、操作合約、顯示使用者資料與 DAO 投票狀況，讓整個系統真的可以跑起來而不是只有合約在鏈上孤零零的，但是前端常常出問題真的很痛苦，所以這次實作也更知道如何除錯

我希望未來可以進一步整合像是 Chainlink 價格餵價、使用 Layer 2（例如 Arbitrum）來省 gas，甚至可以設計治理獎勵分配邏輯來吸引更多人參與提案與投票，讓整個平台更活潑也更去中心化。

操作影片 (其他影片也在這個帳號內): youtube

<https://youtu.be/PWoPNM5vDE4?si=ECFzENLZP42bRoVi>

02

BankV3: 借貸與清算邏輯核心合約



借款人流程與相關 Function

depositCollateral() : 存入 ETH，觸發抵押加入與時間記錄。



流程：

1. 驗證輸入的 ETH 金額是否大於 0。
2. 若是第一次抵押，記錄 collateralJoinedAt[msg.sender] 時間戳。
3. 將傳入的 ETH 累加至 collateral[msg.sender]。
4. 更新全體抵押總額 totalCollateral。
5. 呼叫 _mintReward(...) **!!!!目前先不給存款人：**
 - isLP = false → 表示這是抵押（不是 LP）!!!!可改
 - 根據抵押價值與時間，鑄造 BKT 給用戶。
6. 發出 CollateralDeposited 事件。

```
function depositCollateral() external payable onlyStarted {
    require(msg.value > 0, "Zero collateral");

    // 🕒 如果是首次抵押，記錄加入時間
    if (collateral[msg.sender] == 0) {
        collateralJoinedAt[msg.sender] = block.timestamp;
    }

    collateral[msg.sender] += msg.value;
    totalCollateral += msg.value;

    // ✅ 使用時間加乘獎勵邏輯，isLP = false
    _mintReward(msg.sender, msg.value, false);

    emit CollateralDeposited(msg.sender, msg.value);
}
```

```
mapping(address => uint256) public collateralJoinedAt; // 抵押 ETH 的時間
```

```
mapping(address => uint256) public collateral;
uint256 public totalCollateral;
```

borrow(uint256 amount)事件：Borrowed



功能：

讓用戶從自己的抵押中取出一部分 ETH，但必須確保剩餘抵押仍能覆蓋目前債務（LTV 控制）。



流程：

1. 更新債務：_updateDebt(address user)
2. 計算最大可借金額：getCollateralValueUSD(msg.sender) * LTV / DECIMALS;
3. require不超出上限與流動性
4. 更新系統狀態：增加 debts[msg.sender].principal 與 totalDebtCached
5. 發放借款（從池中轉出 mDAI 給用戶）

```
function borrow(uint256 amount) external onlyStarted {
    _updateDebt(msg.sender);

    uint256 maxBorrow = getCollateralValueUSD(msg.sender) * LTV / DECIMALS;
    require(debts[msg.sender].principal + amount <= maxBorrow, "Exceeds LTV");
    require(amount <= liquidityPool, "Not enough liquidity");

    debts[msg.sender].principal += amount;
    debts[msg.sender].lastAccrued = block.timestamp;
    totalDebtCached += amount;
    liquidityPool -= amount;

    miniDai.transfer(msg.sender, amount);
}
```

_updateDebt(address user)

把使用者目前應該繳的利息「加到本金裡」，也就是把利息實際累加進債務中

⚠ 特別設計 – 防止錯誤：

如果債務變少太多，導致快取值變成負的（不合法），會直接設成 0，這樣最安全不會報錯。

_accruedDebt(address user)

即時計算「目前的總債務」，也就是本金加上從上次計息後累積的利息



功能：

withdrawcollateral(uint256 amount)

讓用戶從自己的抵押中取出一部分 ETH，但必須確保剩餘抵押仍能覆蓋目前債務（LTV 控制）。



流程：

1. 驗證提領金額是否合法 → 不為 0 && 小於等於用戶原本抵押量
2. 執行 _updateDebt(msg.sender) 更新債務（本金 + 利息）
3. 計算新抵押值（newCol）與當前債務（含利息）
4. 驗證：

- 提領後的新抵押價值（用 USD 計算）必須大於等於目前債務 × LTV，否則視為違反抵押比 $\text{newCol} * \text{price} / 1e8 * \text{DECIMALS} \geq \text{debt} * \text{LTV}$

5. 更新 collateral[] 與 totalCollateral

6. 用 transfer 將 ETH 提給使用者

```
function withdrawCollateral(uint256 amount) external onlyStarted nonReentrant {
    require(amount > 0 && amount <= collateral[msg.sender], "Invalid");
    //更新目前的債務（加入利息）
    _updateDebt(msg.sender);
    //計算「提領後剩下的抵押值」與「當前債務（含利息）」
    uint256 newCol = collateral[msg.sender] - amount;
    uint256 debt = _accruedDebt(msg.sender);
    //計算「提領後剩下的抵押值」與「當前債務（含利息）」
    require(newCol * priceOracle.getLatestETHPrice() / 1e8 * DECIMALS >= debt * LTV, "Undercollateralized");
    collateral[msg.sender] = newCol;
    totalCollateral -= amount;
    payable(msg.sender).transfer(amount);
}
```

_updateDebt(address user)

把使用者目前應該繳的利息「加到本金裡」，也就是把利息實際累加進債務中

⚠ 特別設計 – 防止錯誤：

如果債務變少太多，導致快取值變成負的（不合法），會直接設成 0，這樣最安全不會報錯。

_accruedDebt(address user)

即時計算「目前的總債務」，也就是本金加上從上次計息後累積的利息

repay(uint256 amount)事件：RepaidDetailed

✓ 功能：用戶還款（可分期），自動拆解本金與利息。

↺ 流程：避免多收使用者錢
轉入實際款項

1. 若還超過總債務，僅收取應付金額。

2. 償還資金拆分邏輯

a. 本金燒掉（burn）

b. 利息部分分配給：LP（流動性池） && 協議收入
（protocolTokenReserve）

3. 更新債務資訊與利息時間。

4. 加入資金至流動池與協議儲備更新 collateral[] 與
totalCollateral

分拆本金（toBurn）與利息

計算給協議的比例

剩餘部分進流動性池

燒本金 + 更新債務資訊

更新流動性與協議儲備

計算債務

```
function repay(uint256 amount) external onlyStarted {
    require(amount > 0, "Zero repay");

    // 查詢當前債務
    (uint256 principal, uint256 interest) = getUserDebt(msg.sender);
    uint256 userTotalDebt = principal + interest;

    // 如果還太多，就只收最多應還的數量
    uint256 actualAmount = amount > userTotalDebt ? userTotalDebt : amount;

    // 收款（只收合理的 actualAmount）
    miniDai.transferFrom(msg.sender, address(this), actualAmount);

    // 計算要燒掉多少本金
    uint256 toBurn = actualAmount >= userTotalDebt
        ? principal
        : actualAmount > interest
            ? actualAmount - interest
            : 0;

    // 給協議的收入（依利息比例）
    uint256 toProtocol = actualAmount > interest
        ? interest * reserveFactor / DECIMALS
        : actualAmount * reserveFactor / DECIMALS;

    // 剩下給 LP
    uint256 toLP = actualAmount - toBurn - toProtocol;

    // ✓ 如果要燒本金
    if (toBurn > 0) {
        miniDai.burn(address(this), toBurn);
        debts[msg.sender].principal -= toBurn;
        totalDebtCached -= toBurn;
    }

    // 更新利息時間點
    debts[msg.sender].lastAccrued = block.timestamp;

    // 分配資金
    liquidityPool += toLP;
    protocolTokenReserve += toProtocol;

    emit RepaidDetailed(msg.sender, toBurn, toLP, toProtocol);
}
```

03

BankV3: 借貸與清算邏輯核心合約



清算人流程 liquidate()

功能：

當用戶的 Health Factor (HF) 低於安全門檻時 ($HF < 0.825$)，允許任何人清算該用戶的債務，並根據還款金額換得其部分抵押品 (ETH)



流程：

1. 基本限制條件 ($repayAmount \neq 0$ ，更新用戶債務)

2. 檢查是否可清算

- 重新計算總債務 total (含利息)。
- 檢查 Health Factor 是否小於清算門檻 (例如 0.825)。
- 檢查 repayAmount 不可超過總債務。

3. 支付 + 燒毀債務 4. 清算價值

與獎勵計算 5. 估算需沒收多少

ETH 6. 拆分 ETH (清算人 vs 協議)

7. 更新用戶狀態 8. ETH 發

放與儲備

//===清算人===

```
function liquidate(address user, uint256 repayAmount) external onlyStarted nonReentrant {
    require(repayAmount > 0, "Zero repay");
    _updateDebt(user);
```

```
    uint256 total = _accruedDebt(user);
    require(getHealthFactor(user) < LIQUIDATION_THRESHOLD, "Health factor ok");
    require(repayAmount <= total, "Too much");
```

```
    miniDai.transferFrom(msg.sender, address(this), repayAmount);
    miniDai.burnFromBank(address(this), repayAmount); // <- 明確從 Bank 合約自己燒
```

```
    uint256 ethPrice = priceOracle.getLatestETHPrice(); // USD * 1e8
```

```
    // bonus 給清算人，protocolPart 留給協議
```

```
    uint256 bonusUSD = repayAmount * BONUS_PERCENT / DECIMALS;
    uint256 totalUSD = repayAmount + bonusUSD;
```

```
    // seizedETH = (totalUSD * 1e18) / ethPrice
```

```
    uint256 seized = totalUSD * 1e18 / ethPrice;
```

```
    if (seized > collateral[user]) seized = collateral[user];
```

```
    // 精算拆分 (再換算為 ETH 單位)
```

```
    uint256 bonusETH = bonusUSD * 1e18 / ethPrice;    給清算人的
```

```
    uint256 protocolETH = seized > bonusETH ? seized - bonusETH : 0;
```

```
    // 更新抵押資訊
```

```
    collateral[user] -= seized;
```

```
    totalCollateral -= seized;
```

```
    // 更新債務資訊
```

```
    uint256 newPrincipal = total > repayAmount ? total - repayAmount : 0;
```

```
    debts[user].principal = newPrincipal;
```

```
    debts[user].lastAccrued = block.timestamp;
```

```
    totalDebtCached -= repayAmount;
```

```
    // 分配 ETH
```

```
    payable(msg.sender).transfer(bonusETH);
```

```
    protocolETHReserve += protocolETH;
```

```
    emit Liquidated(msg.sender, user, seized);
```

協議收入

04



BankV3: 借貸與清算邏輯核心合約 流動性提供者 (LP) 功能

功能：

允許使用者存入 mDAI，為整個借貸平台提供可貸出的資金，並換得 aMDAI 作為股份憑證；後續可用 aMDAI 贖回資金，並獲得利息收益與治理代幣獎勵



流程：

1. 把 amount 的 mDAI 存入合約（進入流動性池 liquidityPool）
2. 鑄造同樣數量的 aMDAI 作為股份憑證
3. 自動紀錄 LP 加入時間 清算價值與獎勵計算
4. 發放 BKT 獎勵（透過 _mintReward()）拆分 ETH（清算人 vs 協議）



supply() - 使用者提供流動性

```
function supply(uint256 amount) external onlyStarted nonReentrant {  
    require(amount > 0, "Zero supply");  
    miniDai.transferFrom(msg.sender, address(this), amount);  
    aMDAIToken.mint(msg.sender, amount);  
    liquidityPool += amount;  
  
    if (lpJoinedAt[msg.sender] == 0) {  
        lpJoinedAt[msg.sender] = block.timestamp;  
    }  
  
    _mintReward(msg.sender, amount, true); // 傳 isLP = true  
  
    emit Supplied(msg.sender, amount, amount);  
}
```

04

BankV3: 借貸與清算邏輯核心合約 流動性提供者 (LP) 功能

redeem() - 使用者提領流動性



流程：

1. 使用者傳入要燒毀的 aMDAI 數量，依照「佔總池比例」計算可領回的 mDAI 數量鑄造同樣數量的 aMDAI 作為股份憑證
2. 銷毀 aMDAI，實際轉出 mDAI 給用戶

```
function redeem(uint256 amount) external onlyStarted nonReentrant {
    require(amount > 0, "Zero redeem");
    uint256 totalSupply = aMDAIToken.totalSupply();
    require(totalSupply > 0, "No aMDAIToken");

    uint256 redeemAmount = liquidityPool * amount / totalSupply;
    aMDAIToken.burn(msg.sender, amount);
    liquidityPool -= redeemAmount;
    miniDai.transfer(msg.sender, redeemAmount);
    emit Redeemed(msg.sender, redeemAmount, amount); // shares = amount
}
```

贖回比例公式（核心公式）：實時 LP 佔比計算贖回值，確保公平
你可以領回的 mDAI = 你的 aMDAI / 全部 aMDAI × 流動性池總額
使用 aMDAI 作為股份憑證

05

GOVERNOR: DAO 治理架構



借款人流程與相關 Function

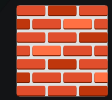


目的：建立去中心化治理機制，讓參與者（BKT 持有者）能

提案（如調整利率、獎勵規則）

投票決策（通過、否決、取消）

排程與延遲執行（透過 Timelock 控制器）



架構組成

用一次完整流程來說明 ----->

05

GOVERNOR: DAO 治理架構



借款人流程與相關 Function

step 1: 部屬合約(deploy.js)

```
// === 3. 部署 BankToken (治理代幣) ===  
const BankToken = await hre.ethers.getContractFactory("BankToken");  
const initialSupply = hre.ethers.parseUnits("1000000", 18);  
const bankToken = await BankToken.deploy(initialSupply);  
await bankToken.waitForDeployment();  
const bankTokenAddress = await bankToken.getAddress();  
console.log(" BankToken:", bankTokenAddress);
```

```
// === 7. 部署 TimelockController ===  
const BankTimelock = await hre.ethers.getContractFactory("BankTimelock");  
const minDelay = 20; // -----  
const proposers = [];  
const executors = [hre.ethers.ZeroAddress];  
const timelock = await BankTimelock.deploy(  
  minDelay,  
  proposers,  
  executors,  
  deployer.address  
);
```

```
// === 8. 部署 Governor ===  
const BankGovernor = await hre.ethers.getContractFactory("BankGovernor");  
const proposalThreshold = hre.ethers.parseUnits("0.1", 18);  
const quorumPercent = 0;  
const governor = await BankGovernor.deploy(  
  bankTokenAddress,  
  timelockAddress,  
  proposalThreshold,  
  quorumPercent  
);
```

部署 BankToken (治理代幣)

發行總量 100 萬顆 BKT，作為 DAO 的投票代幣
是 GovernorVotes 的依據

- Timelock 是 DAO 治理中負責「延遲執行」提案的角色。
- 它確保投票通過後不會立刻生效，給社群時間反應。
- minDelay 是提案排程後到可執行之間的最短延遲。
- proposers 初始為空，但稍後會加入 Governor。
- executors 包含 ZeroAddress 表示任何人都可執行通過的提案。

- ankGovernor 是整個 DAO 的大腦，負責提案、投票與排程。
- 提案門檻 (proposalThreshold)：發起提案至少要有多少票 (BKT)。
- 法定門檻 (quorumPercent)：通過提案的最低票數比例 (%)。
- 與 Timelock 搭配運作，確保治理流程合規。

05

GOVERNOR: DAO 治理架構



借款人流程與相關 Function

step 1: 部屬合約(deploy.js)

```
// === 9. 移交治理與設定 DAO 權限 ===  
await bank.setGovernance(timelockAddress);  
console.log(" Ownership transferred to Timelock.");  
  
const proposerRole = await timelock.PROPOSER_ROLE();  
await timelock.grantRole(proposerRole, governorAddress);  
console.log(" Governor granted proposer role.");  
const executorRole = any  
const executorRole = await timelock.EXECUTOR_ROLE();  
await timelock.grantRole(executorRole, hre.ethers.ZeroAddress);  
console.log(" Anyone granted executor role (ZeroAddress).");
```

```
// === 10. 初始 BKT 分配 ===  
await bankToken.transfer(timelockAddress, hre.ethers.parseUnits("600000", 18));  
await bankToken.transfer(deployer.address, hre.ethers.parseUnits("200000", 18));
```

- BankV3 合約本身包含一個 governance 權限，預設是 deployer。
- 這裡我們 把治理權限交給 Timelock，確保只能透過投票控制平台參數。
- Timelock 使用 角色（Role）權限控制 誰能提案與執行。
- Governor 被授予提案權（Proposer）。
- ZeroAddress 被授予執行權，表示 任何人都可執行通過的提案。

把大部分的治理代幣分配給 Timelock（DAO 國庫）。
剩下的部分分給 deployer 作為初始提案與投票用。

各 ABI json 檔

contract-address.json 存放所有合約地址

```
{  
  "aMDAI.json"  
}  
js app_copy.js  
js app.js  
{  
  "BankGovernor.json"  
}  
{  
  "BankTimelock.json"  
}  
{  
  "BankToken.json"  
}  
{  
  "BankV3.json"  
}  
{  
  "contract-address.json"  
}  
<> index.html  
{  
  "MiniDAI.json"  
}  
{  
  "PriceOracle.json"
```

05

GOVERNOR: DAO 治理架構



借款人流程與相關 Function

step 2: 得到bkt 讓你擁有提案和投票權 方法：提供流動性或抵押(未開放) BankToken (ERC20Votes)

```
_mintReward(msg.sender, amount, true); // 傳 isLP = true
```

step 3: 委託投票權

持有者必須先 委託 (delegate) 投票權給自己或他人，否則沒有投票/提案能力。

bankToken.delegate(account); // 委託投票權給自己

```
//將投票權委託給自己
async function delegateVotes() {
  const tx = await bankTokenContract.delegate(account);
  await tx.wait();
  await updateVotingInfo(); // ← 放這行

  alert("成功委託投票權給自己");
}
```

Step 4：提出治理提案BankGovernor

當你的 BKT 持幣量 $\geq$ 提案門檻 (proposalThreshold)，即可使用 propose() 提出提案。

```
<div>
  <label for="newInterestRate">選擇新利率 (% 年利率) </label>
  <select id="newInterestRate">
    <option value="100">1%</option>
    <option value="300">3%</option>
    <option value="500" selected>5%</option>
    <option value="700">7%</option>
    <option value="1000">10%</option>
    <option value="1500">15%</option>
  </select>
  <textarea id="proposalDesc" placeholder="提案描述"></textarea>
  <button onclick="proposeChangeInterest()">調整利率</button>
</div>
```

```
async function proposeChangeInterest() {
  const rate = parseInt(document.getElementById("newInterestRate").value);
  const desc = document.getElementById("proposalDesc").value;

  const encoded = bankV3Contract.interface.encodeFunctionData("setInterestRate", [rate]);
  const target = await bankV3Contract.getAddress(); // 傳
  const tx = await bankGovernorContract.propose([target], [0], [encoded], desc);
  const receipt = await tx.wait();

  alert("提案成功 ID: " + receipt.logs[0].args.proposalId.toString());
}
```

05

GOVERNOR: DAO 治理架構



借款人流程與相關 Function

step 2: 得到bkt 讓你擁有提案和投票權 方法：提供流動性或抵押(未開放) BankToken (ERC20Votes)

```
_mintReward(msg.sender, amount, true); // 傳 isLP = true
```

step 3: 委託投票權

持有者必須先 委託 (delegate) 投票權給自己或他人，否則沒有投票/提案能力。

bankToken.delegate(account); // 委託投票權給自己

```
//將投票權委託給自己
async function delegateVotes() {
  const tx = await bankTokenContract.delegate(account);
  await tx.wait();
  await updateVotingInfo(); // ← 放這行

  alert("成功委託投票權給自己");
}
```

Step 4：提出治理提案BankGovernor

當你的 BKT 持幣量 $\geq$ 提案門檻 (proposalThreshold)，即可使用 propose() 提出提案。

```
<div>
  <label for="newInterestRate">選擇新利率 (% 年利率) </label>
  <select id="newInterestRate">
    <option value="100">1%</option>
    <option value="300">3%</option>
    <option value="500" selected>5%</option>
    <option value="700">7%</option>
    <option value="1000">10%</option>
    <option value="1500">15%</option>
  </select>
  <textarea id="proposalDesc" placeholder="提案描述"></textarea>
  <button onclick="proposeChangeInterest()">調整利率</button>
</div>
```

```
async function proposeChangeInterest() {
  const rate = parseInt(document.getElementById("newInterestRate").value);
  const desc = document.getElementById("proposalDesc").value;

  const encoded = bankV3Contract.interface.encodeFunctionData("setInterestRate", [rate]);
  const target = await bankV3Contract.getAddress(); // 執行合約地址
  const tx = await bankGovernorContract.propose([target], [0], [encoded], desc);
  const receipt = await tx.wait();

  alert("提案成功 ID: " + receipt.logs[0].args.proposalId.toString());
}
```

執行的合約地址 (如 BankV3)

ETH 數量 (通常為 0)

目標函式與參數

提案說明文字

05

GOVERNOR: DAO 治理架構



借款人流程與相關 Function

拿到一組id

step 5: 投票治理提案



功能目的：參與治理，表決是否通過提案

BankGovernor



操作邏輯

提案發出後，會進入 votingDelay（投票延遲）和 votingPeriod（投票期間），持幣者可進行投票。

```
console.log(" 提交投票:", { proposalId: proposalIdStr, voteType });  
  
const tx = await bankGovernorContract.castVote(proposalId, voteType);  
await tx.wait();
```

// support = 0: 反對, 1: 贊成, 2: 棄權

step 6: 排程（Queue）提案至 Timelock

BankGovernor + BankTimelock



功能目的：提案通過後，進入延遲排程以保護社群

```
const tx = await bankGovernorContract.queue(  
  [targetAddress],  
  [0],  
  [encoded],  
  descriptionHash  
);
```

05 | GOVERNOR: DAO 治理架構



借款人流程與相關 Function

拿到一組id

step 7: 執行提案 (Execute)



功能目的：實際讓提案的動作生效（例如調整利率） BankGovernor + BankTimelock



操作邏輯

延遲時間（minDelay）過後，任何人都可呼叫 execute() 實行提案。

```
const tx = await bankGovernorContract.execute(  
  [targetAddress],  
  [0],  
  [encoded],  
  descriptionHash  
);
```

step 9: DAO 改參數

```
const encoded = bankV3Contract.interface.encodeFunctionData("setInterestRate", [rate]);
```

```
function setInterestRate(uint256 newRate) external onlyGovernance {  
  require(newRate <= 2000, "Too high");  
  interestRate = newRate;  
}
```

```
modifier onlyGovernance() {  
  require(  
    msg.sender == governance ,  
    "!gov"  
  );  
  _;  
}
```